

Physical AI RaccoonBot OpenVLA 파이프라인 확장 과제 보고서

2021741026 송인규

GitHub: https://github.com/Songik40/Raccoonbot_Openvla

1. 과제 개요

KWU-FAIR-LAB의 RaccoonBot OpenVLA 파이프라인을 기반으로 Dataset Extension과 Code Improvement를 수행하였다.

2. Dataset Extension

기존 한계: cylinder 오브젝트만 존재, grasp 단일 태스크, instruction 1종

Extension 내용:

- Orange box 오브젝트 추가 (target_object_box)
- Diverse language instruction 추가: grasp / pick up / grab / lift 4종 템플릿 적용
- MuJoCo 시뮬레이션으로 cylinder 400개 + box 100개 = 총 500 에피소드 생성
- RLDS 포맷 변환 완료 (train 449개, val 50개 / 1개 실패 에피소드 제외)
- TFDS 빌드 완료 (raccoon_pick_place/1.0.0, 290MB)
- 에피소드 시각화 완료 (episode_visualization.png — rollout 주요 프레임 6장 그리드)
- cylinder 400개 + box 100개 통합 데이터셋으로 LoRA 파인튜닝 완료 (500 steps, run: raccoon-diverse-instructions)

3. Code Improvement

- 개선 1 — 7D→4DOF Action Mapping (openvla_server.py)
 - OpenVLA의 7D 출력 [dx, dy, dz, droll, dpitch, dyaw, gripper] 중 실제 로봇 제어에 필요한 [dx, dy, dz, gripper] 4개만 추출하여 반환하도록 수정. 추론 시간도 함께 로깅하여 성능 분석 가능하도록 개선.
 - Before: 서버가 7D action 반환, 클라이언트에서 직접 슬라이싱 처리 필요.
 - After: 서버가 4D action 직접 반환, 추론 시간 로깅 추가 (약 46ms/step).
- 개선 2 — 클라이언트 4D Action 지원 (raccoon_env.py)
 - 기존 7D만 허용하던 execute_delta_action7 함수를 4D도 처리 가능하도록 수정. 서버-클라이언트 간 action 차원 호환성 확보.

- *Before*: `len(action) < 70`이면 `ValueError` 발생.
- *After*: `len(action) >= 40`이면 처리 가능, 4D/7D 모두 호환.
- 개선 3 — Rollout 시각화 (`visualization.ipynb`)
 - MuJoCo rollout 프레임을 mp4로 저장하는 코드 구현. `rollout_box.mp4` 생성 완료 (50 steps, step 35부터 gripper 닫힘 확인).

4. Inference 결과

체크포인트(`fair-lab/opencvla-7b-finetuned-raccoonbot`)를 사용하여 MuJoCo 클라이언트 실행 성공. instruction "grasp the orange box"로 50 steps rollout 완료, `rollout_box.mp4`로 저장. 추가로, 본 과제에서 확장한 통합 데이터셋(cylinder 400 + box 100, diverse instructions)으로 LoRA 파인튜닝을 수행하여 체크포인트를 생성하였다 (`opencvla-7b+raccoon_pick_place+b16+lr-0.0005+lora-r32`, 500 steps).

5. 결론

기존 cylinder grasp 단일 태스크에서 box 오브젝트 추가, 다양한 language instruction 구현, 4DOF action mapping 개선, rollout 시각화까지 파이프라인 전반을 확장하였다.

파인튜닝 과정에서 여러 문제가 있었다. 서버 환경이 PyTorch CUDA 12.1과 CUDA toolkit 11.3이 혼재된 상태라 `bitsandbytes`가 로드되지 않았고, `BNB_CUDA_VERSION=118` 환경변수를 설정해 해결했다. QLoRA와 DDP를 함께 쓸 때 gradient checkpointing 충돌이 생겨 `finetune.py`에 `enable_input_require_grads()`와 `_set_static_graph()`를 직접 추가했다. 공유 A100 서버에서 다른 컨테이너가 GPU를 점유해 OOM이 발생하는 경우도 있었다.

이번 프로젝트는 데이터 수집부터 RLDS 변환, TFDS 빌드, LoRA 파인튜닝까지 전체 파이프라인을 직접 구축하면서 각 단계에서 환경 의존성 문제를 몸으로 익힐 수 있는 좋은 경험이었다.